

MINOSOA

Analyse de ton fichier config/database.php

✓ Tes points forts

Simplicité : Ton code est lisible et va droit au but. Pour un développeur qui doit rapidement comprendre comment se connecter, c'est idéal.

Gestion des exceptions : L'utilisation du bloc try...catch est un excellent réflexe. Cela permet d'éviter qu'une erreur de connexion ne fasse "planter" toute l'application de manière brutale.

Mode d'erreur PDO : Tu as bien configuré ATTR_ERRMODE sur ERRMODE_EXCEPTION. C'est indispensable pour intercepter les erreurs SQL pendant le développement.

Tes points à améliorer

Encapsulation : Tes variables (\$host, \$pdo, etc.) sont définies dans l'espace global. Si tu as d'autres fichiers avec des variables portant le même nom, cela pourrait créer des conflits. C'est pour cela que tes camarades de l'**AI&Dev Academy** utilisent souvent des classes.

Charset manquant : Tu n'as pas précisé charset=utf8 dans ta chaîne de connexion. Sans cela, les caractères spéciaux des noms malgaches pourraient mal s'afficher.

Sécurité de l'affichage : En production, évite d'utiliser echo \$e->getMessage(). Cela pourrait révéler des informations sensibles sur ton serveur à un utilisateur malveillant.

Analyse de ton fichier `models/Contact.php`

✓ Tes points forts

Architecture Objet cohérente : L'instanciation de la base de données dans le constructeur est une excellente pratique qui permet de séparer la logique de connexion de la logique des données.

Sécurité SQL : Tu utilises parfaitement les requêtes préparées avec des marqueurs nommés (:name, :email) pour les méthodes create et delete. C'est la protection standard contre les injections SQL enseignée à l'AI&Dev Academy.

Simplicité du code : Le passage direct d'un tableau associatif à la méthode execute() rend ton code très lisible et moderne.

Format de retour : Utiliser fetchAll(PDO::FETCH_ASSOC) est idéal pour le MVC car cela renvoie des données "brutes" (des tableaux) que la vue pourra manipuler facilement sans dépendre de PDO.

Tes points à améliorer

Gestion de la connexion : Dans ton constructeur, tu fais new Database(). Assure-toi que ta classe Database dans config/database.php possède bien une méthode connect(). Si ton fichier de config est resté en mode procédural (comme le premier que tu m'as montré), ce modèle provoquera une erreur.

Typage des paramètres : Pour plus de rigueur, tu pourrais ajouter le type des arguments, par exemple public function delete(int \$id). Cela évite d'envoyer accidentellement du texte là où on attend un nombre.

Ordre d'affichage : L'ajout de ORDER BY id DESC dans getAll() est un excellent réflexe pour l'expérience utilisateur, permettant de voir les nouveaux contacts en premier.

Analyse de ton ContactController.php

✓ Tes points forts

Gestion des flux (Redirections) : L'utilisation de `header("Location: index.php?action=list")` après un ajout ou une suppression est une excellente pratique. Cela évite le problème du "double envoi" si l'utilisateur rafraîchit sa page.

Simplicité et clarté : Ton code est très lisible. On comprend immédiatement l'enchaînement des actions : le contrôleur reçoit la demande, sollicite le modèle, puis redirige ou affiche la vue.

Délégation au Modèle : Tu n'as aucune logique métier ou SQL ici, ce qui est parfait. Tu te contentes de passer les variables `$name` et `$email` à ton modèle.

Tes points à améliorer

Validation du format : Tu vérifies si les champs sont vides avec `!empty()`, mais comme pour certains de tes camarades, il manque une validation du format de l'email (via `filter_var`).

Sécurité des paramètres : Pour la méthode `delete()`, tu passes `$_GET['id']` directement. Il serait plus prudent de s'assurer qu'il s'agit bien d'un nombre entier en faisant un cast : `(int)$_GET['id']`.

Retour utilisateur : Actuellement, si l'utilisateur oublie de remplir un champ, il est simplement redirigé vers la liste sans savoir pourquoi l'ajout a échoué. Une gestion de messages d'erreur améliorerait l'expérience utilisateur.

Analyse de ton fichier views/contacts.php

✓ Tes points forts

Protection XSS systématique : Tu as utilisé `htmlspecialchars()` pour afficher le nom et l'email. C'est le réflexe de sécurité le plus important pour un développeur front-end, et tu l'as parfaitement intégré.

Expérience Utilisateur (UX) : L'ajout de la boîte de dialogue de confirmation (`onclick="return confirm(...)"`) est une excellente pratique. Cela évite les erreurs de manipulation et rend l'application plus robuste.

Syntaxe PHP propre : L'utilisation de la structure `foreach (...) : et endforeach;` (bien que tu aies utilisé les accolades ici, ce qui fonctionne aussi très bien) est très lisible dans un fichier HTML.

Tes points à améliorer

Sémantique HTML : Il manque les balises `<thead>` et `<tbody>` dans ton tableau. Bien que le navigateur l'affiche correctement, c'est une norme importante pour l'accessibilité et le référencement.

Design et Mise en page : Ton interface est très minimaliste. L'ajout d'un peu de CSS (même simple) permettrait de rendre le tableau plus lisible, par exemple en ajoutant des bordures ou un peu d'espacement (*padding*).

Fiche d'Évaluation : Projet Répertoire MVC

Étudiante : Minosoa

Projet : Gestionnaire de Contacts (Style Minimaliste)

Note Globale : 17 / 20

Détail de l'Évaluation

Critère	Note	Observations
Architecture MVC	5 / 5	Découpage impeccable entre le modèle, le contrôleur et la vue.
Sécurité & Rigueur	4 / 5	Requêtes préparées et protection XSS validées. Attention à la validation de l'email côté serveur.
Qualité du Code (PHP)	4,5 / 5	Code très lisible, moderne et facile à maintenir.
Interface & UX (UI)	3,5 / 5	Fonctionnel et sécurisé, mais mériterait un effort sur l'esthétique visuelle.